SIMATS ENGINEERING

ASSESSMENT TOOL 2 – ANSWERS

Course Name: Artificial Intelligence Course Code: CSA1711

CO2: Search and game-solving techniques – Greedy Search, A*, CSP, Minimax, Alpha-Beta pruning.

Question 1 – Emergency Drone Delivery

i. Greedy Best-First Search behavior: Expands the node with the smallest heuristic value (straight-line distance to goal) only, ignoring the cost already spent. It reacts fast, needs little memory, and suits the drone's urgency. Weakness: it is not optimal and not guaranteed safe — it can commit to a route that looks close but is actually blocked or dangerous, since it ignores real terrain/weather cost and partial updates.

ii. A* behavior: Uses f(n) = g(n) + h(n), balancing the real cost travelled (g) with the estimated remaining cost (h). This lets it account for changing terrain/weather cost while still being guided toward the goal. With an admissible heuristic it is optimal and complete, but a static A* still needs to be re-run (or use an incremental variant) when the map changes.

iii. Impact of heuristic accuracy: Greedy search's quality depends entirely on the heuristic — an inaccurate estimate can send the drone the wrong way with no correction. A* is more robust because g(n) grounds it in real cost: an admissible/consistent heuristic keeps A* optimal; an overestimating heuristic breaks optimality; a weak (underestimating) heuristic only costs extra time, not correctness.

iv. Effect of dynamic blocked paths: Both algorithms are hurt by dynamic changes. Greedy search can get trapped, having already committed to a now-blocked path. A* can recover better because it tracks real cost and can re-expand alternatives, but for real efficiency it needs a dynamic/incremental variant (e.g., D* or Lifelong Planning A*) instead of restarting from scratch each time.

v. Recommendation: A* (ideally its dynamic variant, D*/D*-Lite) is the most suitable choice. It gives an optimal, cost-aware route rather than a merely 'looks close' route, which matters for safety in an emergency, and it can be adapted to replan quickly as blocked paths are reported, satisfying the real-time constraint.

Question 2 – Smart City Traffic Signals

Formulation: This is an optimization problem: state = signal timing configuration across all intersections; objective function = weighted sum of total waiting time, fuel consumption, and congestion (to minimize). Traditional (uninformed) search is inefficient because the state space is combinatorially huge (many intersections × many timing options) and constantly changing, and there is no useful notion of a 'path' to a goal — only a best configuration to find, which unguided search cannot do efficiently.

i. Hill Climbing: Starts from one timing configuration and repeatedly moves to a neighboring configuration with a better score. It is simple and fast but only looks at immediate neighbors with no lookahead, so it can settle on a poor configuration in such a large, dynamic space.

ii. Local maxima, plateaus, ridges: Local maximum: a timing plan where no small change helps, even though a much better city-wide plan exists elsewhere (e.g., optimal for one junction cluster but poor overall). Plateau: many different timing plans score equally, so the algorithm wanders without progress. Ridge: improvement requires changing two or more signals together (e.g., two coupled intersections); single-variable hill climbing cannot make that combined move and stalls.

iii. Simulated Annealing / Genetic Algorithms: Simulated Annealing occasionally accepts a worse configuration (with a probability that shrinks over time) to escape local maxima and plateaus. Genetic Algorithms keep a population of candidate timing plans and use crossover/mutation to explore many regions of the search space in parallel, avoiding early convergence to a poor solution.

iv. Recommendation: A Genetic Algorithm (optionally hybridized with Simulated Annealing) is best, because it scales across hundreds of intersections through parallel population search and can be re-run periodically as traffic patterns shift, giving both scalability and adaptability.

Question 3 – Mars Rover

i. Why an online search agent: The rover has no complete map in advance, so a plan cannot be computed fully offline. It must interleave sensing, computing, and acting — moving, observing the result, and deciding the next action — which is exactly what an online search agent does.

ii. Partial observability and dynamic environment: Limited sensing means hazards outside sensor range are unknown until encountered, risking the rover entering a bad or irreversible situation. A changing environment (shifting dust, lighting, terrain) means previously learned costs can go stale, forcing continual replanning while still acting cautiously to avoid irreversible mistakes.

iii. Building an internal model: As it explores, the rover records visited cells, discovered obstacles, and estimated costs into an internal map (an occupancy-style model), updating heuristic/cost estimates as in Learning Real-Time A* (LRTA*), so future decisions improve with experience.

iv. Online vs. uninformed vs. informed search: Uninformed search (BFS/DFS) needs the full state space and explores blindly. Informed search (A*) needs a known map and heuristic ahead of time. Online search has neither luxury — it acts with partial information, updates its model as it goes, and may revisit states, interleaving planning and execution rather than planning fully in advance.

v. Recommendation: An online informed strategy such as LRTA* is most suitable: it uses heuristic guidance where available, updates itself as new terrain is discovered (adaptability), and takes cautious, reversible-first steps to protect the rover (safety) under uncertainty.

Question 4 – University Exam Timetabling (CSP)

Variables: Each exam/course to be scheduled (e.g., E1, E2, …, En).

Domains: For each exam, the set of valid (time slot, exam hall) pairs it could be assigned.

Constraints: (1) No two exams taken by the same student share a time slot. (2) Each hall's capacity/availability is not exceeded in any slot. (3) Certain subjects must be scheduled before others (precedence). (4) Assigned faculty must be free at that slot. (5) Special accommodation exams need a compatible room/slot (e.g., extra time, separate hall).

ii. Backtracking search: Assigns a slot/hall to one exam at a time, checking it against all constraints from prior assignments; if a constraint is violated it backtracks and tries the next domain value for the previous exam, continuing until a complete valid timetable is found or the search space is exhausted.

iii. Constraint propagation (forward checking): After each assignment, forward checking removes now-invalid slot/hall options from the domains of unassigned exams that conflict with it (e.g., a shared-student clash). This detects dead ends early and prunes the search tree, which is essential as the number of courses/students grows.

iv. Real-world challenges and best strategy: Challenges include rapidly growing domains/constraints with scale, competing constraints (faculty, room, precedence, accommodations) and fairness. The best strategy is backtracking with forward checking plus variable-ordering heuristics (Minimum Remaining Values, Degree heuristic) — this combination prunes the search efficiently and is the standard approach used in real university timetabling systems.

Question 5 – RTS Game AI (Minimax & Alpha-Beta)

i. Minimax modeling: The game is modeled as an adversarial, two-player zero-sum tree: the AI (maximizer) picks moves that maximize its outcome, assuming the opponent (minimizer) always responds optimally against it. Values propagate bottom-up from terminal/cutoff states to choose the best worst-case move.

ii. Computational challenges: Game trees grow exponentially, O(b^d) with branching factor b and depth d. RTS games have large branching factors and long horizons, so evaluating the full tree within the game's strict time limits is computationally infeasible, and storing it demands large memory.

iii. Alpha-Beta pruning: Alpha-Beta tracks the best value the maximizer can already guarantee (alpha) and the best value the minimizer can already guarantee (beta) while traversing the tree, and stops exploring (prunes) any branch that cannot change the final decision once alpha ≥ beta. This does not change the result, but in the best case cuts time complexity from O(b^d) to O(b^(d/2)), allowing much deeper search in the same time.

iv. Evaluation function: A weighted combination of factors that indicate how favorable a state is: unit count/strength difference, resources/economy, territory or map control, technology/upgrade level, and positional advantage. These are used because full game trees can't be searched to terminal states in real time, so a heuristic estimate at the cutoff depth is needed.

v. Recommended strategy: Use iterative deepening Alpha-Beta search with a strict time cutoff, combined with good move ordering (searching likely-best moves first to maximize pruning) and a well-tuned evaluation function. This lets the AI search as deep as time allows while guaranteeing it always returns a move within the real-time limit.